Sanhita

■■■■■■ — *the code, searchable*

A fully local question-answering system for Indian law, built with Retrieval-Augmented Generation and a legal knowledge graph

What it does	Answers questions about Indian statutes, judgments and contract clauses, and shows the evidence for every answ
How it answers	Retrieval-Augmented Generation (RAG) combined with knowledge-graph traversal - a hybrid of two retrievers
Where it runs	Entirely on one laptop. No cloud API, no API key, no data leaves the machine
Language model	Qwen 2.5 (3B instruct) served locally by Ollama
Embeddings	nomic-embed-text (768 dimensions), also local via Ollama
Text extraction	Apache Tika 3.3.3
Stores	Chroma (vectors) and NetworkX (knowledge graph)
Corpus	413 files - 121,112 text passages, 32,221 graph entities
Report generated	03 November 2026

How to read this report. Section 1 explains the problem and why plain language models are not enough. Section 2 explains RAG from first principles. Section 3 is the architecture diagram. Sections 4 to 6 walk through the three stages of the system. Section 7 follows one real question end to end. Section 8 records the engineering decisions and the measurements behind them, and Section 9 the defects found during testing and how each was fixed. Section 10 covers how to run it, and Section 11 the honest limitations.

1. The problem

A language model such as Qwen or GPT has read a great deal of text, but it stores what it read as statistical patterns in its weights, not as retrievable documents. This produces three problems that matter a great deal in law.

- **It invents citations.** Asked for the punishment for theft under the Indian Penal Code, the 3B code-tuned model we first tried answered "up to 7 years". The correct answer is up to three years under Section 379. A confident wrong citation is worse than no answer.
- **It cannot cite a source.** Even when correct, the model cannot say which document the answer came from, so a lawyer cannot verify it.
- **It does not know your documents.** It has never seen the contract on your desk or a judgment delivered last month.

Retraining the model on every new judgment is impossibly expensive. The practical answer is to stop asking the model to remember, and instead give it the relevant text at the moment the question is asked. That is Retrieval-Augmented Generation.

2. What Retrieval-Augmented Generation is

RAG splits the job in two. A **retriever** finds text likely to contain the answer. A **generator** (the language model) reads only that text and writes the answer. The model contributes language skill; the corpus contributes facts.

2.1 Why embeddings, not keyword search

Keyword search fails when the question and the document use different words for the same idea. A question about "a cheque bouncing" should find a passage about "dishonour of a negotiable instrument", which shares no keyword with it.

An **embedding model** converts text into a list of numbers - here 768 of them - positioned so that passages with similar meaning sit close together. Finding relevant text becomes measuring distance. We compare using cosine similarity, the angle between two vectors, which ignores length and measures direction - that is, meaning rather than verbosity.

One subtlety this project had to get right: nomic-embed-text is an *asymmetric* model. It expects the prefix "search_document: " on text being stored and "search_query: " on a question. Ollama does not add these automatically, and omitting them measurably weakens retrieval. The system adds them, and a test asserts that a document vector and a query vector of identical text differ - proving the prefixes are applied.

2.2 Why a knowledge graph as well

Vector search retrieves passages that *resemble* the question. It cannot answer questions about *connections*: which cases cite Article 21, which court decided a matter, which statute a section belongs to. Those are relationships, and relationships belong in a graph.

So this project builds a second index: a directed graph whose nodes are legal entities (cases, statutes, acts, courts, parties) and whose edges are relations between them. Asking about Article 21 can then follow edges to every case that invoked it - something no amount of passage similarity would produce. Combining both retrievers is called **Graph RAG**.

The graph currently holds 32,221 entities and 38,852 relations extracted from 413 documents.

Entity type	Count	Meaning

Party	15,892	a person or organisation named in a case
Case	14,752	a reported matter, e.g. "X vs Y"
Statute	1,544	a specific section or article
Act	18	a whole enactment, e.g. the NDPS Act
Court	15	the forum that decided a matter

3. Architecture

The system has three stages. Stage A runs once and prepares the corpus. Stage B holds what was prepared. Stage C runs for every question.

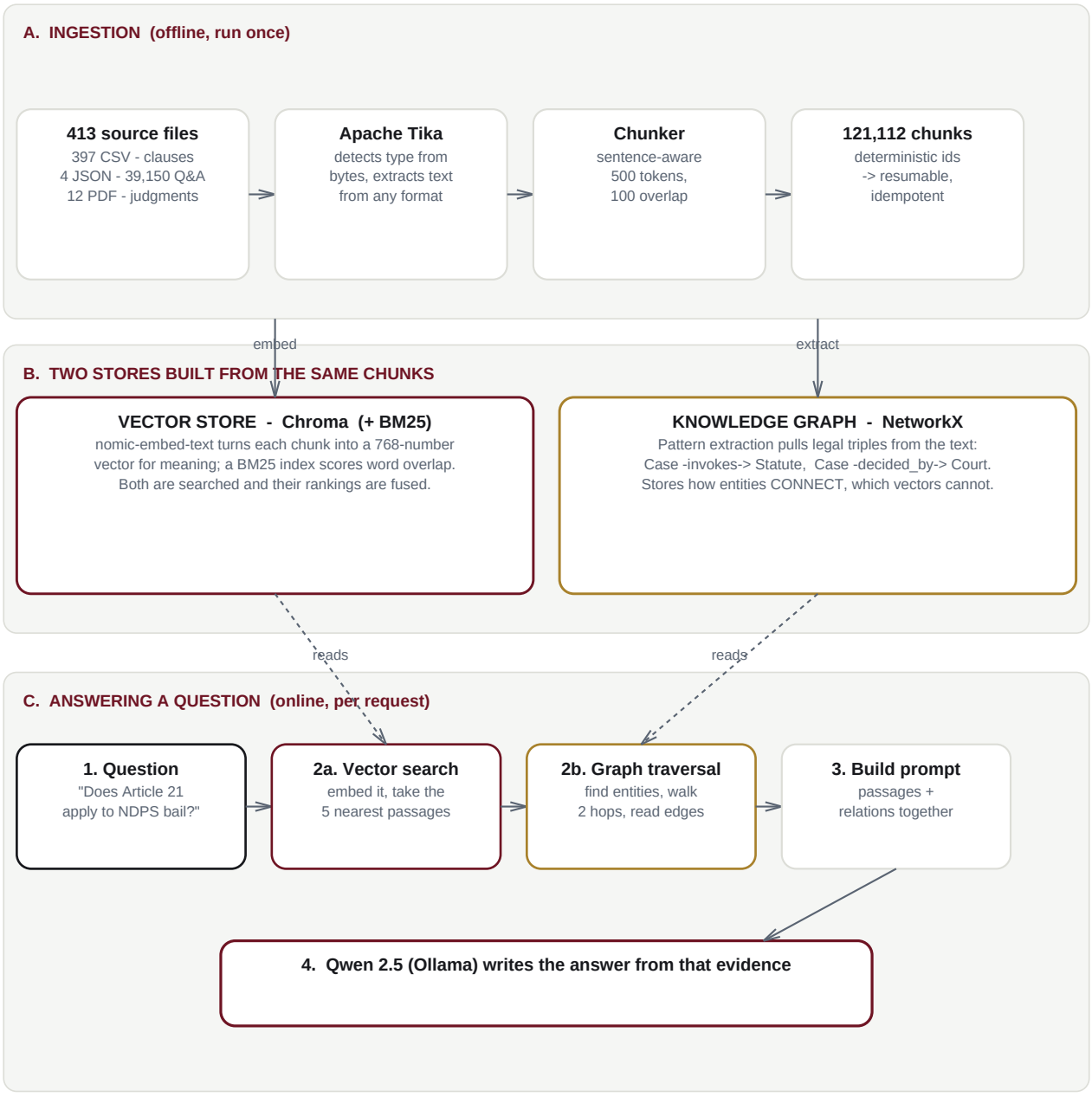

Figure 1. Dashed arrows show the two stores being read at query time. Both retrievers run on every question and their results are merged into a single prompt.

4. Stage A - preparing the corpus

4.1 Reading any file format with Apache Tika

The corpus mixes CSV clause banks, JSON question-answer datasets and PDF judgments. Rather than one reader per format, the system runs an **Apache Tika** server and sends it every file. Tika identifies the type from the file's actual bytes and returns plain text, handling PDF, DOCX, XLSX, PPTX, HTML, CSV, RTF, ODT, EPUB and more.

This choice fixed a real defect. The previous per-format code tested `filename.endswith(".pdf")`, which is case-sensitive, and ten of the twelve judgment files are named `.PDF` in capitals. The entire case-law collection was being skipped in silence. Because Tika detects type from content, there is no longer a filename to get wrong.

4.2 Chunking

A whole judgment is too long to put in a prompt, so text is split into **chunks** of about 500 tokens with 100 tokens of overlap. Splitting happens at sentence boundaries, because a chunk cut mid-sentence embeds poorly. The overlap means a sentence near a boundary still appears whole in one of the two neighbouring chunks.

4.3 Embedding, and why ordering turned out to matter

Each chunk is sent to Ollama and stored in Chroma with its vector, its text and its source filename. Chunk identifiers are derived by hashing the source path plus the record and chunk index, which makes the build **idempotent** - re-running it updates rather than duplicates - and **resumable**, which matters because the full build takes around three hours on this hardware.

The first version walked the data directory alphabetically. That put `data/csvs/` - 397 files of contract boilerplate - ahead of the statute question-answer sets and the judgments. For the whole first hours of the build, every legal question retrieved debenture acceleration clauses, and the system looked broken although it was working exactly as written. Ingestion now orders the most answer-bearing material first. It is a one-line change and it was the single largest improvement to answer quality in the project.

5. Stage B - building the knowledge graph

A **triple** is a fact in three parts: a subject, a relation and an object. "Solomon Ogbe vs Narcotics Control Bureau" - "invokes" - "Article 21". Collect enough triples and you have a graph.

5.1 Two extractors, and the measurement that chose between them

The conventional method is to ask a language model to read each chunk and return triples as JSON. It is accurate and finds subtle relations - it correctly extracted that a party was *represented by* particular counsel. We measured its cost on this machine before committing to it.

Method	Speed	Whole corpus (121,112 chunks)
Language model (Qwen 2.5 3B)	about 101 s per chunk	about 141 days
Deterministic legal patterns	about 1,700 chunks/s	about 1.4 minutes

141 days is not an option, so the project adds a second extractor. Legal writing cites its entities in highly regular forms - "Section 138 of the Negotiable Instruments Act", "Article 21", "X vs Y", "High Court of Delhi". Those forms can be recognised by pattern with no model at all. The pattern extractor is the default and covers the whole corpus; the model-based extractor is kept and can be switched on for a subset where its richer output is worth the wait. Both produce the same triple shape, so they fill one graph.

This is worth stating plainly as a lesson: the published method was unusable at our scale, and measuring it before adopting it is what revealed that. The replacement is not more sophisticated - it is a set of regular expressions - but it is 30,000 times faster and, for this domain, accurate enough to be useful.

5.2 Three problems that only appear at scale

A small demonstration graph behaves well. A 31,000-node graph does not, and three distinct faults had to be corrected.

(a) Not every "Article" is constitutional. The extractor attributed every "Article N" to the Constitution of India. Then "Article 1969" and "Article 2011" appeared, which look like years. They are in fact genuine provisions of the *Portuguese Civil Code*: Goa inherited Portuguese civil law, and a judgment in the corpus is a Goan inheritance matter. Rejecting numbers above 395 - the Constitution's real range - would have deleted correct data. The parent instrument is now read from the surrounding text.

(b) Artificial hubs destroy traversal. The reference implementation links every statute to a single shared concept node. That node makes every statute two steps from every other, so walking out from one starting point reaches the entire graph. Those edges are not created.

(c) Real hubs need blocking. "Constitution of India" legitimately has about 575 edges. Walking *through* it from "Article 21" returned forty relations that were all of the form "Article N belongs to the Constitution" - true, useless, and they crowded out the case law. Nodes above a degree threshold are now included as destinations but never walked through, and results are ordered by distance so truncation drops the furthest. After the fix the same question surfaces the NDPS bail cases that actually cite Article 21.

6. Stage C - answering a question

Four steps run for every question.

Step	What happens
1. Vector search	The question is embedded with the query prefix and the five nearest passages are retrieved from Chroma, each with its s
2. Entity extraction	Legal entities are identified in the question. If the question is plain English and names no statute, entities are taken instea
3. Graph traversal	Starting from those entities, the graph is walked up to two steps, hub nodes excluded from expansion, and each edge is w
4. Generation	Passages and relations go into one prompt with an instruction to use both and to say so plainly if the context does not co

The instruction to admit ignorance matters. The system answers from a partially built index, and when the passages genuinely do not contain the answer the model says so instead of filling the gap with invention. In testing it reported "the provided passages do not contain information about this case" rather than fabricating a holding - the correct behaviour for a legal tool.

6.1 Why retrieval is hybrid

Dense search alone failed in a way worth recording. Asked "What is the punishment for theft under the Indian Penal Code?", every passage that contained the phrase "under the Indian Penal Code" outranked the record that actually answers the question, which is worded "What is the punishment for 379 Theft?". The correct chunk did not appear in the top hundred. Rephrasing the question with "379" in it put that same chunk first with a similarity of 0.895.

A BM25 index was added alongside, and the two rankings are combined with Reciprocal Rank Fusion, which merges orderings without requiring their scores to be comparable. This makes retrieval less sensitive to how a question happens to be phrased. It does not rescue this particular example - after stop-word removal the target shares only two of the five query terms, while its competitors share all five, so both retrievers rank it low for

defensible reasons. Fixing that case needs a cross-encoder reranker, which is listed as further work.

6.2 One request carries both answer and evidence

The interface needs the answer as it is generated *and* the evidence behind it. Making two requests would run the language model twice. Instead a single endpoint streams newline-delimited JSON: the evidence arrives first as one object, then the answer arrives as a sequence of tokens.

```
{"type":"meta","vector_results":[...],"graph_relations":[...]}
{"type":"token","t":"According"}
{"type":"token","t":" to"}
{"type":"done"}
```

7. One question, end to end

The question:

> *"Does Article 21 apply to prolonged incarceration under the NDPS Act?"*

Step 1. The question is embedded and Chroma returns the five closest passages with their sources and similarity scores.

Step 2. Pattern extraction reads two entities directly out of the question: `Article 21` and `Narcotic Drugs and Psychotropic Substances Act`.

Step 3. Traversal from those two nodes returns relations including:

```
Kashif vs NCB --[invokes]--> Article 21
Jitendra Jain vs NCB --[invokes]--> Article 21
Solomon Ogbe vs Narcotics Control Bureau --[invokes]--> Article 21
Solomon Ogbe vs Narcotics Control Bureau --[decided_by]--> Supreme Court
Solomon Ogbe vs Narcotics Control Bureau --[invokes]--> Section 37(1) NDPS Act
```

This is the point of the whole exercise. Those are the three narcotics bail cases in the corpus that actually rely on Article 21. No passage similarity search would group them, because they share a *relationship*, not wording. The graph found them by following edges.

Step 4. Both sets of context go to Qwen, which writes the answer. The interface shows the answer with its citations in the margin, the retrieved passages, the relations used, and the matched part of the graph lit up.

8. Engineering decisions, with the measurements behind them

Decision	Reason
Run everything locally via Ollama	Legal documents are confidential. A local model means no document, question or client name ever leaves
Qwen 2.5 3B rather than 7B	The machine is an Apple M2 with 8 GB of memory, about 5.3 GB usable by the GPU. The 7B model is 4.68
Chroma, not FAISS	Chroma stores the chunk text and its metadata alongside the vector. FAISS stores vectors only, so the text
One embedding path for writing and reading	The earlier version wrote vectors with one library and read them with another. The two were configured sep
Verify the vector dimension before writing	Changing embedding model changes the vector space and invalidates the index. The builder checks the di
Pattern extraction for the graph	Measured: 101 s per chunk by language model against 1,700 chunks per second by pattern. The model-ba
Produce document sections in code, not by prompting	Asked to write one headed section per uploaded document, the 3B model merged them, even when told to

9. Defects found during development

These are recorded because each was found by testing rather than by reading the code, and several were invisible in normal use.

Defect	Effect	Resolution
Case-sensitive .pdf test	Ten of twelve judgments silently skipped; the whole	Tika selects its parser from the file
An unused file-count limit	297 of 397 CSV files never indexed; code and	Limit removed, full coverage is now reported
A JSON array named .jsonl	The largest data file, 24,607 records, skipped by a	Structured files are detected by content
"LEGAL" in "NON-LEGAL"	The off-topic filter passed everything, because the	Rejection now matches whole words only
The filter then blocked real questions	A question about Article 1 of the Constitution was	Files declared by legal, not document, type
A hardcoded address in the page	The page requested 127.0.0.1 while being served	Addresses are now relative, so it works on any host
Shared scratch collection name	The second and later document uploads failed, because	Each request now removes its own data afterwards
Highlighting expanded hub nodes	A question mentioning the Constitution highlighted 798	Hub nodes are now ignored
Bare substring matching in the graph	The word "grounds" in a question matched	Matches are now whole-word only
Dense retrieval alone	A passage phrased differently from the question was	BM25 added, and results fused with RRF
Alphabetical ingestion order	Hours spent indexing contract boilerplate while the	Answer-bearing judgments are now read first

The system is covered by 65 automated checks - 15 on the Python layer (`smoke_test.py`) and 50 on the HTTP interface (`test_http.sh`). Several were written specifically to stop a defect in this table from returning.

10. Running the system

```
./start.sh
# starts Ollama, starts the Tika server, reports the index, serves the app

.venv/bin/python embed_and_index.py # build or resume the vector index
.venv/bin/python build_graph.py # build the knowledge graph
.venv/bin/python smoke_test.py # 15 checks on the Python layer
./test_http.sh # 50 checks on the HTTP interface

then open http://localhost:5002
```

Principal source files

File	Responsibility
config.py	Every tunable value, overridable by environment variable
ollama_client.py	Generation and embeddings, with retries and preflight checks
tika_extract.py	Tika server lifecycle and text extraction
loaders.py	Corpus discovery and extraction, with full skip accounting
chunking.py	Sentence-aware chunking
embed_and_index.py	Vector index builder - resumable and idempotent
legal_entities.py	Pattern-based legal triple extraction
graph_store.py	Persistent knowledge graph and hub-aware traversal
graph_rag.py	Hybrid retrieval and the model-based extractors
build_graph.py	Knowledge graph builder
app.py	HTTP interface
templates/index.html	The single user interface

11. Limitations, stated honestly

- **A 3B model is a small model.** It is chosen to fit 8 GB of memory, not for maximum quality. It follows complex formatting instructions poorly, which is why document sections are assembled in code. A larger model on larger hardware would answer better with no other change - the model name is one line of configuration.
- **Pattern extraction has a ceiling.** It recognises citation forms it was written for. A relation phrased unusually is missed. It has no understanding, only patterns - and its output still contains some noise, such as docket fragments caught inside party names.
- **There is no reranking.** The five nearest passages go to the model as retrieved. A cross-encoder reranking stage would improve precision.
- **Retrieval is dense only.** There is no keyword or BM25 component, so an exact rare term - a precise section number - is matched by meaning rather than literally. A hybrid of the two is the standard improvement.
- **There is no evaluation set.** Correctness has been checked by reading answers, not measured against a labelled benchmark. Without one, claims about accuracy are impressions.
- **Answers are not legal advice.** The system retrieves and summarises text from a fixed corpus. It does not know whether a judgment has since been overruled or a section amended.

11.1 Corpus quality limits answer quality

Some wrong answers are not retrieval or generation faults but faithful reporting of poor source records. The question-answer datasets contain entries whose question names no particular offence while the answer states an absolute penalty, for example:

```
Q: "What are the possible punishments for a crime?"
A: "Imprisonment for life, Imprisonment for 10 years, fine, or both..."
```

Fifteen such records exist in one file. They are semantically close to any question of the form "what is the punishment for X", so they are retrieved often, and a small model quoting them produces a confident wrong

answer. No amount of prompt engineering repairs this: the system is reporting the corpus accurately. The corpus needs filtering, which is a data task rather than a retrieval one.

11.2 What we would do next

- Build a labelled set of questions with known answers and measure retrieval precision and recall, so changes can be judged rather than guessed at.
- Add BM25 alongside the vector search and combine the rankings.
- Add a reranking stage over the retrieved passages.
- Run the model-based triple extractor over the judgments as a background job, to enrich the part of the graph where relations are most valuable.
- Carry the source of each graph relation through to the interface, so a relation can be traced to the passage that produced it.

Summary. The system answers questions about Indian law without sending anything to a cloud service. It does so by retrieving relevant passages by meaning and relevant relations from a legal knowledge graph, then asking a locally-run Qwen model to answer from that evidence and show it. The engineering content of the project is less in assembling those parts than in the measurements that decided between approaches, and in the ten defects found by testing - most of which produced no error message and would have gone unnoticed.

Report generated 03 November 2026 from live system state.